

PHYS-467 Assignment 1

Deadline: Monday 31/10/2022 at 18:00

Instructions

The homework is structured in two parts.

In the Coding Part, you are required to implement some algorithms, and you will be evaluated on the correctness of your code. You will be graded on the code that is online on github at the time of the submission deadline. Code that is not uploaded to github or too late, will not count towards your grade. Note that there is no slack to the deadline, so be sure to push your code timely.

In the Theory Part, you will have to solve some simple theoretical problems/questions. In this case, the deliverable is a pdf file. Please list other students that you consulted with during the completion of this assignment at the beginning of that file (this will not penalize you but it is a good practice to acknowledge other people's help).

1 Coding Part

For this part you will need to reuse the environment `ml4phys` you created in assignment 0. Your environment will additionally need the package 'pandas' over the one in assignment 0. There are two ways to do this:

- You can reuse your environment from assignment 0 by activating it (using `conda activate ml4phys`) and then writing `conda install pandas`.
- Delete the environment with `conda env remove -n ml4phys` (while the environment is deactivated!) and then reinstalling it via `conda env create -n ml4phys --file environment.yml`

Follow the link to the github assignment provided on moodle in 'Assignment 1', and select your email from the list, just like you did for assignment 0. If you have problems with the submission of this code assignment, make sure that you completed assignment 0 successfully and understood the tools introduced there.

Before starting the assignment read the `README.md` file on github. Remember to run your tests locally. You can generate all plots by running `python main.py` manually. Do not change the plots names!

In this exercise the goal is to go through the complete pipeline of implementing a machine learning model. In point 1 you will load the data and normalize it. Then point 2 and 3 will require you to implement a function to train a linear predictor (with either no, ℓ_1 or ℓ_2 regularization) and to evaluate it on new points, and finally execute a complete fit $\rightarrow$ predict $\rightarrow$ test cycle. In point 3 you will plot the learning curve of the model for different values of the regularization parameter. Point 5 requires you to study the behavior of the trained weights as a function of the regularization strength. You will then select the best ℓ_2 regularization in point 6, using a validation set. Finally, in point 7, the input space will be augmented with new features and you will train the linear model again, and study again how the trained weights change with the regularization.

1. **Preprocessing (4 P).** Load the dataset from the files in the `data` folder, normalize the targets y linearly to have standard deviation 1 and print some of summary of the dataset. Fill the functions `load_and_normalize_data` and `data_summary` in `main.py`.

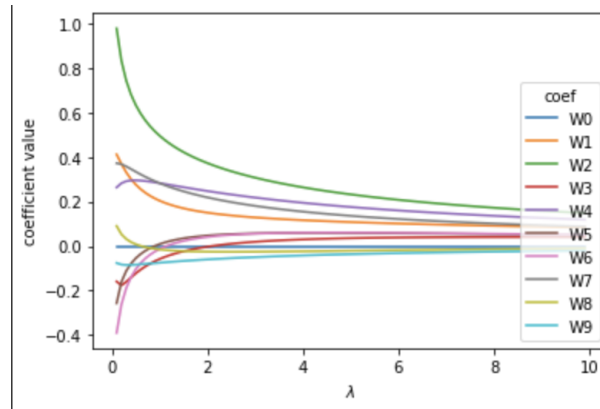


Figure 1: Example of plot of weight coefficients vs regularization strength λ .

2. **Fitting a Linear Regression with varying regularizers (8 P).** Complete `fit_linear_regression` which takes in a dataset plus options for the regularizer and returns the coefficients and intercept of a linear model fitted to this dataset with either no, ℓ_1 or ℓ_2 regularizers. Do not fit an intercept.
3. **Helper functions (8 P).** Complete `predict` for predicting the target for a dataset for a given linear model and `mse` to compute the mean squared error¹ between two target vectors. In `fit_predict_test` collect the results of one experiment using the previously coded methods for fitting and predicting an estimator into a dictionary with the results.
4. **Learning Curves (10 P).** Plot the learning curve (i.e. test error vs number of training samples) and the training error curve (i.e. train error vs number of training samples) of the least squares linear regression with (ℓ_2) regularization λ . Put $\alpha = n/d$ on the x axis of your plots and the train/test error on the y-axis. To do so, complete `plot_dataset_size_vs_mse`, edit the part below `if __name__:` ... and execute `python main.py` to generate the train/test learning curves. Make three plots: one with $\lambda = 0$ and $\alpha \in [2, 6]$, one with $\lambda = 10^{-3}$ and $\alpha \in [0, 2.5]$ and one with $\lambda = 10$ and $\alpha \in [0, 2.5]$.
5. **Weight Coefficients (10 P).** Training on the entire training set, vary the regularization strength λ of the ℓ_2 regularizer and plot the components of the trained weights multiplying x_0, x_3, x_7, x_8 ² as a function of λ (see Figure 1 for an example). To do so, complete `plot_regularizer_vs_coefficients` and insert the correct indices in the part below `if __name__:` ..., so that you can produce the plot.
6. **Hyperparameter Optimization (10 P).** We now want to find the best regularization parameter for the ridge regression. We do this by optimizing the regularizer with respect to the validation dataset. That is for different regularizers do training on the training dataset, find the validation error on the learned model and then select the parameter with the best validation error on its model. Plot your results and also return the generalization error and regularization parameter for this best model. Do the above by completing `optimize_lambda`. Use this function in the part below `if __name__:` ..., for the first $n = 50, 150$ datapoints of the training dataset by searching over the space of regularizer parameters that seems reasonable to you.
7. **Feature Engineering (6 P).** Augment your data by adding the features $x_0^2, \dots, x_{99}^2$ to your training set. Complete `add_poly_features` and use it to transform the validation and test set in the part below `if __name__:` Using Lasso (ℓ_1) regression with regularization strength λ , plot the weight components that multiply $x_0, x_2, x_6, x_7, x_{29}^2, x_{54}^2$ ³ for a model trained with the ℓ_1 regularization. Compare it to a model with ℓ_1 regularizer trained on the original data (non-augmented), $\lambda \in [0.0001, 1.2]$.

¹Remember to normalize the training error, i.e. using the factor $1/(\text{\#training samples})$ at the front.

²With indexing starting from zero.

³With indexing starting from zero.

2 Theory Part

Answer the following questions by writing no more than 5 lines of text per question. Submit a single pdf file containing your answers on the assignment page on moodle. In the following d will be the dimension of the data and n will be the size of the training set.

1. **(6 P.)** You are training a ridge regression model with zero regularization. How does the training loss behave for $n < d$? Explain your answer.
2. **(7 P.)** Let $X \in \mathbb{R}^{n \times d}$ be the training data matrix and $y \in \mathbb{R}^n$ be the labels. Under what condition on X and y is ridge regression with zero regularization able to exactly fit the training points?
3. **(8 P.)** You train ridge regression several times with fixed n and increasing regularization λ . How do you expect the training error to depend on λ ? Express the training error at $\lambda = \infty$ in terms of the labels of the training set $y = (y_1, y_2, \dots, y_n)$. Explain your reasoning